

Rapport Digitala Projekt 5p

Ljusföljande tvåhjulig robot med radiolänk

<[U1tr4-L4zEr 3k]>

Grupp 10

Erik Adlers (d03ea@student.lth.se)

David Eklund (cim03de6@student.lth.se)

Daniel Rasmusson (cim03dr4@student.lth.se)

Department of Information Technology

Lund Institute of Technology

Lund University



Abstract

The goal of the project was to build a two-wheeled mobile robot with the ability to detect, turn towards and keep the distance from, a light source. The actuators were two 5V dc-motors controlled with PWM and the sensors were three daylight-filtered photo transistors. The robot was also supposed to have a 433MHz wireless transmitter, sending updates to a computer using the UART-protocol. The control algorithm and the radio link's protocol were implemented on a Atmel AVR Mega 16 microcontroller. The robot was also equipped with a 2x16 character LCD-display from Sharp to facilitate debugging and give status reports during operation.

Innehållsförteckning

RAPPORT DIGITALA PROJEKT 5P	1
ABSTRACT	2
INNEHÅLLSFÖRTECKNING	3
INLEDNING	4
TEORI	4
PROCESSOR	4
LJUSSENSORER	4
MOTORSTYRNING	4
RADIOLÄNK	4
LCD-DISPLAY	5
KONSTRUKTION OCH UTFÖRANDE	5
ROBOTKONSTRUKTION	5
ANALOGA/DIGITALA KRETSAR	5
PROGRAMMERING	5
RESULTAT OCH SLUTSATS	6
BILAGOR	7
BILAGA 1 KRETSSCHEMAN	7
BILAGA 2 KÄLLKOD	9

Inledning

Projektets syfte var att bygga en tvåhjulig mobil robot som kunde följa och hålla avståndet till en ljuskälla. Hjulens drevs av två 5V DC-motorer med PWM-styrning och ljuskällan detekterades med hjälp av tre dagsljusfiltrerade fototransistorer. Roboten skulle också ha en radiolänk som med hjälp av UART-protokollet skickade uppdateringar till en dator. Protokollet och regulatören för styrningen implementerades på en Atmel AVR Mega16. Roboten utrustades också med en display för att underlätta felsökning och för att ge statusinformation under körning.

Teori

Processor

Systemet byggdes på en RISC-baserad Atmel AVR Mega16 enchipdator med en intern klocka på 8MHz. I denna fanns såväl PWM-generator, AD-omvandlare och en UART-krets som vi behövde för att realisera vår robot. Processorn programmerades i C med hjälp av AVR Studio 4 som använder gcc-kompilatorn. Den kompillerade maskinkoden skickades över till processorn från datorn med hjälp av en JTAG, som även tillät felsökning under körning.

Ljussensorer

För att detektera ljuskällan använde vi tre stycken OP705A, som är dagsljusfiltrerade fototransistorer. Dessa seriekopplades sedan med motstånd till jord för att kunna uttrycka ljusstyrkan i spänning. Mellan fototransistor och resistor kopplades den positiva ingången till en differentialförstärkande OP-koppling med en CA3160 OP-förstärkare. Till differentialförstärkarens negativa ingång kopplades en potentiometer för att kunna kalibrera förstärkarens nolla på utgången. Efter förstärkarkopplingen kopplades till sist ett lågpasfilter för att ta bort brus, framförallt vid 100Hz, orsakat av nätspänningen. (se figur 1, bilaga 1)

Motorstyrning

Motorstyrningen byggdes med hjälp av två 5V likströmsmotorer, styrda av en PWM-signal via en H-brygga. En PWM-signal är en styrsignal där en periodtid delas mellan hög och låg signal. Om man skickar en hög signal under en stor del av perioden kommer medelvärdet närma sig hundra procent. Mega16-processorn kan utnyttja sina inbyggda räknarregister för att generera upp till 4 PWM-signaler. En H-brygga är en krets som dels kan välja i vilken riktning spänningen ska gå genom motorn och dels gör det möjligt att ge en mycket högre effekt till motorn än vad utgångarna på processorn klarar av att driva.

Radiolänk

För radiokommunikationen använde vi AVR Mega16-processorns inbyggda UART-krets som vi sedan skickade genom en 433MHz radiolänk till ett mottagarkort som via en MAX233-krets kopplades på en datorns seriella ingång. För datorn byggde vi sedan ett program i Java som kunde ta emot de skickade signalerna enligt standard ASCII-tabell. För att filtrera bort störningarna i radiolänken skickades ett antal start-

och sluttecken vid varje överföring. Den seriella länken kördes i asynkront i 2400baud/s, med en stoppbit och ojämn paritetskontroll. (se figur 2, bilaga 1)

LCD-Display

På roboten satt också en Sharp LM162, som är en 2x16 tecken alfanumerisk LCD-display. Denna kan konfigureras och skrivas till via en 8-bitars parallell buss enligt standard ASCII-tabell.

Konstruktion och utförande

Konstruktionen kan delas in i tre delar; byggandet av roboten, konstruktion av de analoga och digitala kretsarna och programmering av enchipsdatorn. Dessa delar av konstruktionen utfördes till stora delar parallellt. Roboten växte fram allt eftersom vi byggde fungerande analoga kretsar och subrutiner till processorn.

Robotkonstruktion

Konstruktionen består av modulen med likströmsmotorerna i botten, fastskruvade på ett tomt kretskort. På det tomma kretskortet monterades också ett stöd längst bak, bestående av en lätttrullande metallkula. På det första kortet monterades sedan, via distanser två kretskort till. På mittenkortet satt fototransistorerna med tillhörande kopplingar och på det översta kortet satt processor, LCD-display, radiolänk och H-brygga.

Analoga/Digitala kretsar

De största enskilda kretarna är fototransistorkopplingarna med differentialförstärkare, kalibreringspotentiometrar och filter. Dessa byggdes parallellt med resten av konstruktionen och sitter på ett eget kretskort. Till motorstyrningen behövdes förutom H-bryggan också en inverterare för att välja riktning på motorerna. LCD-displayen kopplades direkt till processorn med en parallell 8-bitars buss och ett par ytterligare bitar för att, bland annat, välja om data eller instruktioner skulle skrivas. Radiolänken är mycket enkel och krävde bara den logiska signalen från processor, matningsspänning och en enkel antenn bestående av en kabel. På datorsidan användes förutom radiomottagaren också en MAX233-krets för att uppfylla serieportsstandarden. Matningsspänningen till radiosändaren kopplades via en spänningsregulator för att eliminera överspänningen från batteriet. (se figur 2 & 3, bilaga 1)

Programmering

Programmeringen av enchipsdatorn bestod nästan uteslutande av att bygga subrutiner för robotens olika funktioner. Subrutinerna delades upp i c-bibliotek bestående av en c-fil med kod och en h-fil med funktionsdefinitioner. Initieringsbiblioteket innehöll initieringsfunktioner till alla andra subrutinsbibliotek. Här ställdes alla I/O-portar till in eller utsignaler, och LCD-displayen, PWM:en, AD:n och UART:en initierades och konfigurerades. C-bibliotek med subrutiner för att styra riktning och styrka på PWM-signalerna, skicka data till radiolänken via UART, ta in värden från fototransistorerna

via AD-omvandlaren, skriva till I/O-portarna och flytta markören och skriva ASCII-tecken eller hexadecimala tal på LCD-displayen skrevs också. Till sist skrevs ett bibliotek med regulatorer för att rikta roboten mot ljuskällan och för att hålla avståndet till densamma. Regulatorerna var proportionella och styrdes med konstanta referenssignaler och är-värden konstruerade från sensorvärdena enligt följande:

$$Y_{\text{vinkel}} = (s_2 - s_3) * 255 / (s_1 + s_2 + s_3)$$

$$Y_{\text{avstånd}} = 255 - (s_1 + s_2 + s_3)$$

Där s_1 - s_3 är sensorvärdena mellan [0..255] för mitten-, höger- och vänstersensorn resp. Dessa ekvationer bygger på att sensorerna är lineära när ljuskällan förflyttas i sidled framför dem, och att sensorerna överlappar så att sidosensorerna börjar ge utslag när mittsensorn ger maximalt utslag (ljuskällan är rakt framför roboten). (se bilaga 2 för källkod)

Resultat och slutsats

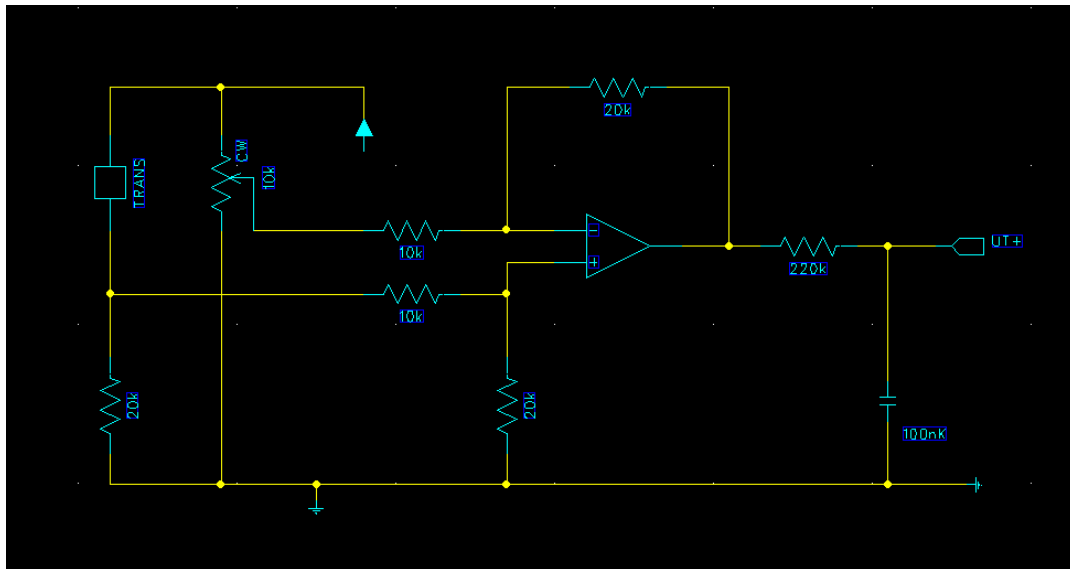
Roboten uppfyller specifikationen och även kravet för trådlös radiokommunikation till en dator. Roboten kan rikta in sig mot en ljuskälla och hålla ett givet avstånd från denna.

Radiolänken fungerar, d.v.s. att roboten kan skicka meddelanden till en dator som är ansluten med mottagaren.

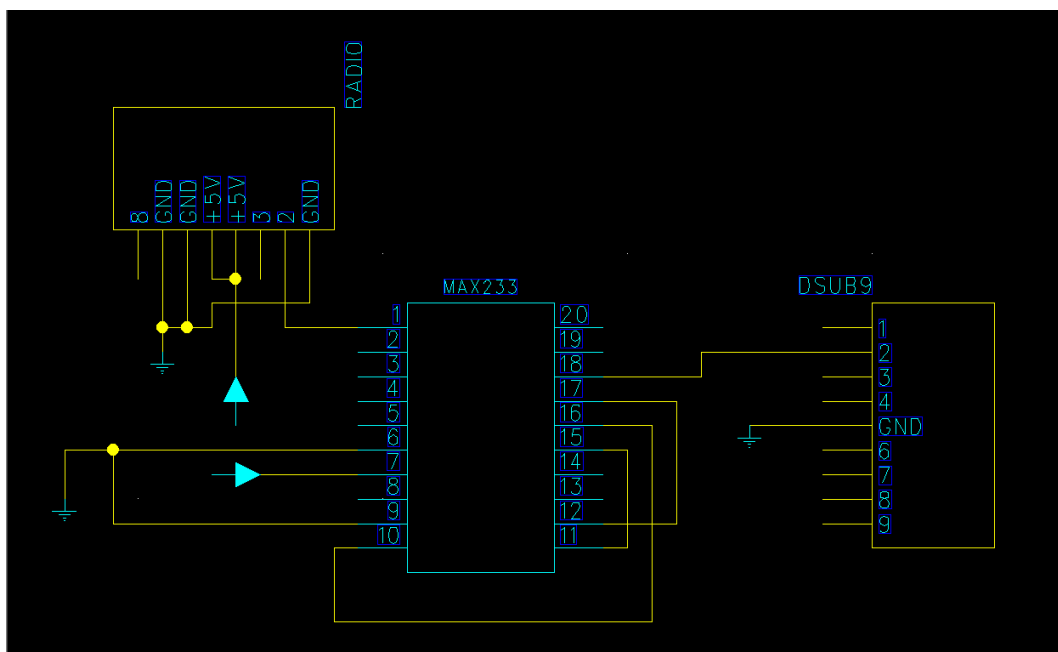
Det faktum att vi använder P-regulatorer för både vinkel- och avståndsreglering resulterar i ett kvarstående fel. Detta fel är dock inte större än de andra felkällorna i roboten, t.ex. sensorer, brus och avrundningsfel.

Bilagor

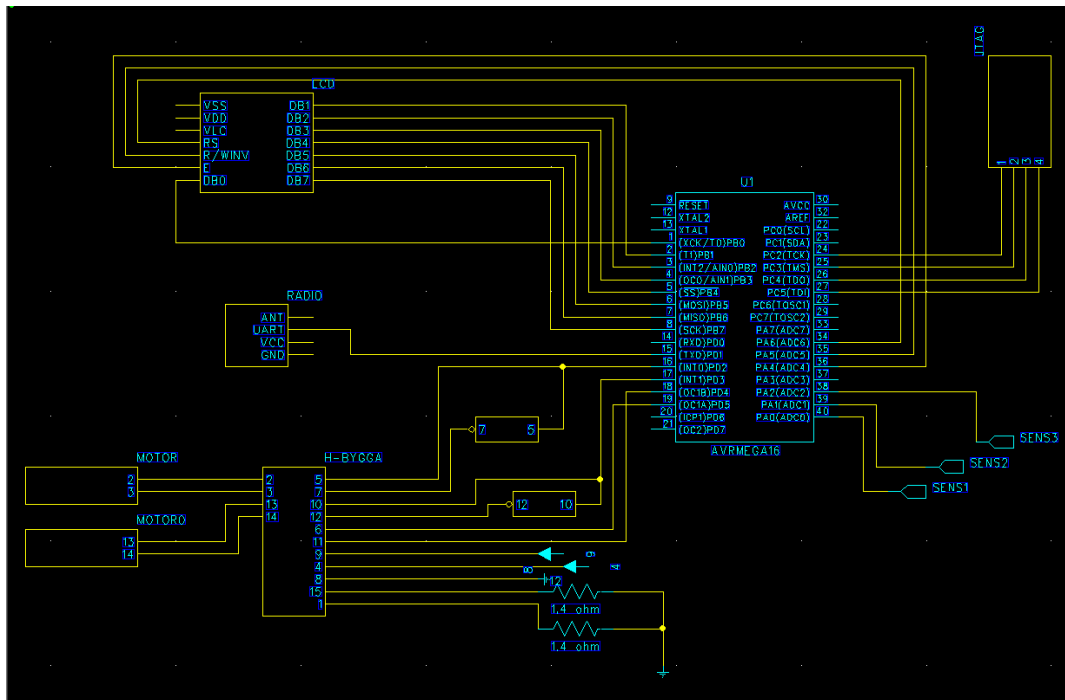
Bilaga 1 Kretsscheman



Figur 1: Kretsschema differentialförstärkare



Figur 2: Kretsschema radiomottagare



Figur 3: Kretsschema roboten

Bilaga 2 Källkod

main.c

```
#include <avr/io.h>
#include <util/delay.h>
#include "init.h"
#include "io.h"
#include "lcd.h"
#include "ad.h"
#include "pwm.h"
#include "servo.h"
#include "usart.h"
#include "pregul.h"

int main(void)
{
    //massor emd många inits
    init_io();
    init_lcd();
    init_ad();
    init_pwm();
    init_usart();

    pwm_set('A',0);
    pwm_set('B',0);

    lcd_write_string(0,0,"Status:");

    while(1)
    {
        usart_send_string("Z");
        p_regulator();

    }//while(1)

    return 0;
}

//Main
```

ad.c

```
#include <avr/io.h>
#include "ad.h"
#include "io.h"

//Returnerar en char med värdet från ad-omvandling på
pinne pin
unsigned char ad_read(unsigned char pin)
{
    ADMUX += pin; //Välj
    ADCSRA += 0x40;
    while(ADCSRA&0x40);
    ADCSRA = ADCSRA | 0b00010000;
    ADMUX -= pin;
```

```

        return ADCH;
    }

init.c

#include <avr/io.h>
#include "delay.h"
#include "io.h"
#include "lcd.h"
//Initierar IO-portarna
void init_io()
{
    //sätt alla pinnar till utportar
    DDRA = 0xF0;
    DDRB = 0xFF;
    DDRC = 0xFF;
    DDRD = 0xFF;
    PORTA = 0x00;
    PORTB = 0x00;
    PORTC = 0x00;
    PORTD = 0x00;
}

void init_lcd()
{
    delay_ms(20);
    lcd_write_inst(0b00000001);
    delay_ms(4);
    lcd_write_inst(0b00111100);
    delay_ms(1);
    lcd_write_inst(0b00001111);
    delay_ms(1);
    lcd_write_inst(0b00000110);
    delay_ms(1);
    lcd_write_inst(0b00000001);
    delay_ms(4);
}

void init_ad()
{
    DDRA = DDRA & 0xF0;
    ADMUX = 0b01100000;
    ADCSRA = 0b10000111;
}

void init_pwm()
{
    TCCR1A = 0xA2;
    TCCR1B = 0x13;
    //TCCR1C = 0x00;
    ICR1H = (1250>>8);
    ICR1L = 1250 & 0xFF;
    OCR1AH = 0;// (250 >> 8);
    OCR1AL = 0;//(250 & 0xFF) ;

    OCR1BH = 0;//(500 >> 8);
    OCR1BL = 0;//(500 & 0xFF) ;
}

```

```

void init_usart()
{
    unsigned int baud = 207;
    UBRRL = (unsigned char) (baud>>8);
    UBRRL = (unsigned char) baud;
    UCSRB = (1<<TXEN);
    UCSRC = (1<<URSEL) | (3<<UCSZ0 ) | (3<<UPM0);
}

```

io.c

```

#include <avr/io.h>

```

```

//Sätter pinnen i position pos på porten port till val
void io_set(char port, char pos, char val)

```

```

{
    switch (port) {
        case 'A':
            if(val)
            {
                PORTA |= (1<<pos);
            }
            else
            {
                PORTA &= ~(1<<pos);
            }
            break;
        case 'B':
            if(val)
            {
                PORTB |= (1<<pos);
            }
            else
            {
                PORTB &= ~(1<<pos);
            }
            break;
        case 'C':
            if(val)
            {
                PORTC |= (1<<pos);
            }
            else
            {
                PORTC &= ~(1<<pos);
            }
            break;
        case 'D':
            if(val)
            {
                PORTD |= (1<<pos);
            }
            else
            {
                PORTD &= ~(1<<pos);
            }
    }
}

```

```

}

```

```

} //io_set

//Returnerar värdet på pinnen pos på port port
char io_get(char port, char pos)
{
    switch (port)
    {
        case 'A': return ((0x01 << pos) & PINA)>>pos;
        case 'B': return ((0x01 << pos) & PINB)>>pos;
        case 'C': return ((0x01 << pos) & PINC)>>pos;
        case 'D': return ((0x01 << pos) & PIND)>>pos;
        default: return 0;
    } //switch
} //io_get

```

lcd.c

```

#include <avr/io.h>
#include <util/delay.h>
#include "lcd.h"
#include "io.h"

//Skriver data (ett ASCII-teckan) till displayen
void lcd_write_data(unsigned char data)
{
    io_set('A', PA6, 1);
    PORTB = data;
    io_set('A', PA4, 1);
    io_set('A', PA4, 0);
    io_set('A', PA6, 0);
}

//Skriver i bytes instruktion till displayen
void lcd_write_inst(unsigned char data)
{
    PORTB = data;
    io_set('A', PA4, 1);
    io_set('A', PA4, 0);
}

//Skriver värdet av b till displayen hexadecimalt [0..FF]
void lcd_write_hex(unsigned char b)
{
    if ((b >> 4) > 9)
        lcd_write_data('A' + ((b >> 4) - 10));
    else
        lcd_write_data('0' + (b >> 4));
    if ((b & 0x0f) > 9)
        lcd_write_data('A' + ((b & 0x0f) - 10));
    else
        lcd_write_data('0' + (b & 0x0f));
}

//Skriver en sträng av chars till displayen
void lcd_write_string(unsigned char rad,unsigned char
plats, char *data)
{

```

```

    int i;

    lcd_set_cursor(rad,plats);

    _delay_ms(1);
    for(i = 0; i< strlen(data); i++)
    {
        _delay_ms(1);
        lcd_write_data((unsigned char)(data[i]));
    }
}

//Sätter markören till raden rad [0..1] och positionen
till pos [0..15]
void lcd_set_cursor(unsigned char rad,unsigned char pos)
{

    lcd_write_inst(pos + rad*0x40+0x80);

}

```

pregul.c

```

#include <util/delay.h>
#include "pregul.h"
#include "lcd.h"
#include "ad.h"
#include "pwm.h"
#include "usart.h"

char active = 0;

//Beräknar motorsignaler från sensorvärden och
referensvärden
void p_regulator()
{
    signed int i, tmp1, tmp2, tmp3;
    double diff, add, fi, r, Pfi, Pr, u_A, u_B;
    double ref_r, ref_fi, K_r, K_fi;

    //Sätter parametrar för regulatorn

    ref_r = 70;
    K_r = 2.5;

    ref_fi = -30;
    K_fi = 0.4;

    //Hämtar sensorvärden
    tmp1 = ad_read(0x00);
    tmp2 = ad_read(0x01);
    tmp3 = ad_read(0x02);

    //Deloperationer
    diff = tmp2 - tmp3;
    add = tmp1 + tmp2 + tmp3;
}

```

```

r = 255 - add;
fi = diff*(255/add);

Pfi = K_fi*(ref_fi - fi);
Pr = K_r*(r - ref_r);
if(Pr > 160)
    Pr = 160;
else if(Pr < -160)
    Pr = -160;

//Beräknar utsignaler

u_A= Pfi + Pr;
if(u_A > 255)
    u_A = 255;
else if(u_A < -255)
    u_A = -255;

u_B= -Pfi + Pr;
if(u_B > 255)
    u_B = 255;
else if(u_B < -255)
    u_B = -255;

//Skickar utsignaler till PWM:en
if(add>15)
{
    pwm_set('A', (int)(u_A));
    pwm_set('B', (int)(u_B));
    if(!active)
    {
        usart_send_string("Status: On\n");
        lcd_write_string(0,8, "On ");
        lcd_write_string(1,0, "Target aquired");
        active = 1;
    } //if
} else
{
    if(active)
    {
        usart_send_string("Status: Off\n");
        lcd_write_string(0,8, "Off");
        lcd_write_string(1,0, "No target      ");
        active = 0;
    }
    pwm_set('A', 0);
    pwm_set('B', 0);
}
}

```

pwm.c

```

#include <avr/io.h>

//Sätter utsignal till motorerna

```

```

void pwm_set(char channel, signed int duty)
{
    int sign;
    double tmp;

    //Separerar tecken och magnitud av duty
    if(duty < 0)
    {
        sign = 0;
        tmp = duty;
        tmp *= -1.1;
        duty = tmp;
    } else
        sign = 1;

    if(duty > 255)
        duty = 255;

    duty *=4;

    //Ställer in PWM:en
    switch (channel)
    {
        //Höger motor
        case 'A':
            io_set('D',PD2,sign);
            /*tmp = duty;
            tmp *= 0.9;
            duty = tmp;*/
            OCR1AH = (duty >> 8);
            OCR1AL = (duty & 0xFF) ;
            break;

        //Vänster motor
        case 'B':
            io_set('D',PD3,sign);
            OCR1BH = (duty >> 8);
            OCR1BL = (duty & 0xFF) ;
            break;
    }
}

```

usart.c

```

#include <avr/io.h>
#include <string.h>
#include "usart.h"

//Skickar en seriell byte mha USART
void usart_transmit(unsigned char data)
{
    while (!(UCSRA & (1 << UDRE))) {}

    UDR = data;
}

```

```
void usart_send_string(char *string)
{
    int i;

    for(i = 0; i<= 20; i++)
        usart_transmit('Y');

    for(i = 0; i<= strlen(string); i++)
        usart_transmit(string[i]);

    for(i = 0; i<= 10; i++)
        usart_transmit('Z');
}
```